Proiect de Programare / C++

- Sistem platformă cursuri online -

Student 1: ALEXANDRU-GABRIEL Butoi
Student 2: GABRIEL-VLAD Stana

I. Enunţ

Studentul 1 se ocupă de gestionarea platformei (aplicaţia `./bin/app1`):

- adăugare, ştergere şi modificare cursuri (titlu, descriere, preţ)
- adăugare şi ştergere lecţii dintr-un curs
- vizualizarea cursurilor existente şi a lecţiilor lor

Studentul 2 se ocupă de utilizatori şi de interacţiunea cu platforma (aplicaţia `./bin/app2`):

- înregistrarea şi ştergerea utilizatorilor
- înscrierea şi dezinscrierea la un curs
- marcarea lecţiilor ca finalizate sau nefinalizate
- vizualizarea progresului la cursurile la care e înscris

Pe lângă cele două aplicaţii din linia de comandă, proiectul include şi o **interfaţă grafică** (`app_gui`), construită cu ImGui + SDL2 + OpenGL, care oferă aceleaşi funcţionalităţi printr-o fereastră cu două tab-uri.

II. Structura datelor folosite de echipă

Vom folosi clasele:

- **Lectie**: `string id_lectie, string titlu, int durata_minute`
- **Curs** (compoziţie cu `Lectie` – o lecţie nu există fără cursul ei):
  `string id_curs, string titlu, string descriere, double pret, Lectie[] lectii`
- **Utilizator**: `string id, string nume`
- **Inscriere** (asociere între un `Utilizator` şi un `Curs`):
  `string id_user, string id_curs, string[] lectii_finalizate`

Generarea ID-urilor. ID-urile cursurilor, lecţiilor şi utilizatorilor sunt numerice şi se generează automat (următorul număr liber). S-a ales această abordare în locul unor ID-uri de tip UUID deoarece aplicaţiile sunt folosite din linia de comandă, unde un ID scurt (`1`, `2`, `3`) este mult mai uşor de tastat decât un şir lung. La ştergerea unui utilizator, ID-urile rămase sunt reechilibrate (`1, 2, 3, ...`), iar înscrierile din `inscrieri.txt` sunt actualizate automat.

Relaţii între clase:

- `Curs` – `Lectie`: **compoziţie** (lecţiile sunt deţinute de curs)
- `Inscriere` – `Curs` / `Utilizator`: **asociere** (prin ID)

III. Structura fişierelor

Fişierele de date se află în directorul `data/`. Fiecare înregistrare se citeşte rând cu rând, până la sfârşitul fişierului (nu există un număr total de înregistrări la începutul fişierului).

cursuri.txt

```
<id_curs1> <titlu_curs1> <descriere_curs1> <pret_curs1>
<numar_lectii_curs1>
<id_lectie1> <titlu_lectie1> <durata_minute1>
<id_lectie2> <titlu_lectie2> <durata_minute2>
...
<id_curs2> <titlu_curs2> <descriere_curs2> <pret_curs2>
...
```

inscrieri.txt

```
<id_user1> <id_curs1> <numar_lectii_finalizate>
<id_lectie_finalizata1> <id_lectie_finalizata2> ...
<id_user2> <id_curs2> <numar_lectii_finalizate>
...
```

(Linia cu lecţiile finalizate apare doar dacă numărul lor este mai mare ca 0.)

utilizatori.txt

```
<id_user1> <nume_utilizator1>
<id_user2> <nume_utilizator2>
...
```

IV. Interacţionarea cu executabilele

Important: toate executabilele se rulează din directorul rădăcină al proiectului, astfel încât căile către `data/` să fie corecte.

Aplicaţia 1 (Student 1) – gestiune cursuri şi lecţii

```
./bin/app1.exe vizualizare_cursuri
   - afiseaza toate cursurile de pe platforma

./bin/app1.exe adaugare_curs <titlu> <descriere> <pret>
   - adauga un curs nou (ID generat automat)

./bin/app1.exe stergere_curs <id_curs>
   - sterge un curs impreuna cu lectiile lui

./bin/app1.exe modificare_curs <pret|titlu|descriere> <id_curs> <valoare_noua>
   - modifica pretul, titlul sau descrierea unui curs

./bin/app1.exe adaugare_lectie <id_curs> <titlu> <durata_minute>
```

```
   - adauga o lectie la un curs existent (ID generat automat)

./bin/app1.exe stergere_lectie <id_curs> <id_lectie>
   - sterge o lectie dintr-un curs
```

Aplicaţia 2 (Student 2) – utilizatori, înscrieri şi progres

```
./bin/app2.exe vizualizare_utilizatori
   - afiseaza toti utilizatorii inregistrati

./bin/app2.exe inregistrare_utilizator <nume>
   - inregistreaza un utilizator nou (ID generat automat)

./bin/app2.exe stergere_utilizator <id_user>
   - sterge un utilizator si reechilibreaza ID-urile

./bin/app2.exe vizualizare_cursuri_inscrise <id_user>
   - afiseaza cursurile la care utilizatorul e inscris

./bin/app2.exe inscriere <id_user> <id_curs>
   - inscrie utilizatorul la un curs

./bin/app2.exe dezinscriere <id_user> <id_curs>
   - dezinscrie utilizatorul dintr-un curs

./bin/app2.exe finalizare_lectie <id_user> <id_curs> <id_lectie>
   - marcheaza o lectie ca finalizata

./bin/app2.exe nefinalizare_lectie <id_user> <id_curs> <id_lectie>
   - marcheaza o lectie ca nefinalizata

./bin/app2.exe vizualizare_progres <id_user> <id_curs>
   - afiseaza cate lectii a terminat utilizatorul din totalul cursului
```

Aplicaţia grafică (app_gui)

```
./bin/app_gui.exe
```

Deschide o fereastră cu două tab-uri:

- **Cursuri** – adăugare, editare şi ştergere de cursuri şi lecţii
- **Inscrieri** – selectarea unui utilizator, înscriere/dezinscriere la cursuri, marcarea lecţiilor ca finalizate şi o bară de progres

V. Detalii de implementare

- **Clase şi obiecte**: `Curs`, `Lectie`, `Inscriere`, `Utilizator`, plus clasele de serviciu.
- **Containere STL**: `std::vector` pentru toate colecţiile, plus algoritmi din `std::ranges`.
- **Supraîncărcarea operatorilor**: `operator«` pentru afişarea fiecărei clase de model.
- **Template**: clasa `Repository<T>` (citire/scriere generică din fişier) şi funcţiile `nextId<T>` / `findById<Vec>`.

- **Smart pointers**: fiecare serviciu deţine repositoarele prin `std::unique_ptr`, eliberând memoria automat.
- **Validare argumente**: fiecare comandă verifică numărul de argumente şi validează datele (preţ pozitiv, ID-uri existente, câmpuri ne-goale).